

AI Infrastructure Resource Ledger: The Reality Check

Thursday, 5. March 2026

Engineer(s): Callie Jones

Date: 3/5/26

Project Title: Vision-Grounded Distance Estimation

Phase 1: The Calendar (Working Backward)

If you are still writing your training loop on your Drop-Dead Date, your project will fail.

1. Final Demo / Presentation Date: 4/28?
2. Days required for Evaluation & Report: 14 days.
3. Estimated Training Run Duration: ~2 days.
4. DROP-DEAD TRAINING START DATE: 4/12 (*Subtract Line 2 & 3 from Line 1*)

Phase 2: The Stack & Compute Target

1. Base architecture backbone, include encoders etc., add as many lines as you need:
Vision encoder for object detection and depth estimation. YOLOv8-n for tennis ball detection, depth estimation network (ex. MiDaS / DPT-Small), fusion MLP head that combines bounding box size and depth features to estimate metric distance.

Param Count: ~0.05 Billion (50 Million)

2. Target Compute Hardware:
[] Local / Lab Node
[x] Cloud Rental (e.g., Lambda/AWS)
3. Specific GPU(s): NVIDIA A100 40GB ?
Total VRAM Available: 24 - 40 GB ?

Phase 3: The Memory Math (The VRAM Ledger)

Reference: 1B Parameters $\approx$ 2GB (Weights) + 2GB (Gradients) + 12GB (AdamW Optimizer States)

1. Model Weights (BF16): 0.1 GB
2. Gradients (BF16): 0.1 GB
3. Optimizer States: 0.6 GB
4. KV Cache & Activations Reserve: 4 GB
5. TOTAL REQUIRED VRAM: ~4.8 GB

The Strategy: Is your Total Required VRAM > Total Available VRAM?

☐ Yes ☒ No

If Yes, what is your exact mitigation strategy? ☐ FSDP / ZeRO-3 (Sharding across multiple GPUs)

☐ LoRA / QLoRA (Freezing base weights, avoiding optimizer state explosion)

☐ I am shrinking my model selection.

☐ Something else (explain):

Phase 4: The Data Ingress (The I/O Bottleneck)

1. Dataset Size: ~12 GB / 20K ? Total Samples

2. Format:

☒ Raw JPEGs/PNGs/trajectories/etc. in a folder (High I/O Risk)

☐ WebDataset / Sharded .tar

☐ Something else

3. Storage Location:

☒ Local NVMe SSD ?

☐ Network Drive

☐ S3/GCS Bucket

☐ Something else

If using raw images or other large data quantities over a network or standard SSD, how will you prevent your dataloader from starving your GPUs?

Potentially use PyTorch multi-worker dataloading, allowing prefetching or possibly caching. Also controlling image resolution by possibly resizing images.

Phase 5: Red Team Audit (Partner Review)

Reviewing Engineer: Kali Hamilton

Look at your partner's ledger. Find the fatal flaw. Are they underestimating training time? Do they have a 40GB model fitting into 24GB of VRAM? Are they trying to load 2 million JPEGs sequentially?

The Fatal Flaw / Biggest Risk:

Finalizing architecture design & tools to be used such as simulating data.

Phase 6: The 72-Hour Contract

What exact, granular engineering task will you complete in the next 72 hours to unblock these risks? (e.g., "Write dummy dataloader script to verify throughput," "Run a 10-step overfitting batch to check VRAM usage.")

Deliverable by Monday: _____

Finalize architecture & tools and also an experiment training run at a small scale to confirm the setup is correct.